

Politechnika Świętokrzyska w Kielcach Wydział Elektrotechniki, Automatyki i Informatyki	
Podstawy Programowania 2 - Projekt Informatyka - I rok, Rok akademicki - 2025/2026	
Temat projektu: Przychodnia	Wykonali: Semen Rotbart Andrii Riabchenko

1. Opis tematyki projektu

Naszym zadaniem było stworzenie od podstaw w języku C systemu do obsługi przychodni lekarskiej. Program pozwala na pełne zarządzanie pacjentami i lekarzami oraz posiada działający moduł rejestracji wizyt. Najbardziej zależało nam na tym, żeby aplikacja sama pilnowała harmonogramu - jeśli lekarz jest zajęty, nie można dopisać do niego drugiego pacjenta na ten sam termin. Dodatkowo wszystkie wprowadzone dane zapisują się do plików tekstowych, więc po restarcie programu nic nie ginie.

2. Użyty język programowania, biblioteki, IDE, OS

- **Język programowania:** C
- **Wykorzystane biblioteki:** Standardowe biblioteki języka C `<stdio.h>`, `<stdlib.h>`, `<string.h>`, `<stdbool.h>`
- **Środowisko programistyczne:** CLion
- **System budowania:** CMake
- **System operacyjny (OS):** Windows 11

3. Instrukcja kompilacji i obsługi

Jak odpalić projekt: Dzięki użyciu CMake, najprościej jest otworzyć folder Przychodnia w CLionie i użyć przycisku Run. Można też skompilować go ręcznie z konsoli. Program uruchamia się w oknie terminala. Jak używać? Aplikacja ma bardzo proste menu tekstowe. Nawigacja polega na wpisaniu odpowiedniej cyfry z klawiatury i zatwierdzeniu klawiszem Enter. Gdy chcemy skończyć pracę, wybieramy opcję wyjścia w menu głównym - wtedy program automatycznie zapisuje wszystkie zmiany do plików .txt.

4. Zrzuty ekranu

```
D:\projects\pp2\Przychodnia\cmake-build-debug\Przychodnia.exe
Baza danych zaladowana pomyslnie.
```

```

MENU GLOWNE
1. Lekarze
2. Pacjenci
3. Wizyty
0. Wyjdz z programu
Wybierz opcje:
```

```
D:\projects\pp2\Przychodnia\cmake-build-debug\Przychodnia.exe
Baza danych zaladowana pomyslnie.
```

```

MENU GLOWNE
1. Lekarze
2. Pacjenci
3. Wizyty
0. Wyjdz z programu
Wybierz opcje:1
```

```

ZARZADZANIE LEKARZAMI
1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot
Wybierz opcje:|
```

Dodawanie lekarzy

Lekarz #2

Tytuł zawodowy:*Lekarz*

Imie:*Semen*

Nazwisko:*Rotbart*

PESEL:*123123123*

Numer PWZ:*1232*

Specjalizacja:*Lekarz*

E-mail:*rotbartsemen@gmail.com*

Telefon:*1232132*

Godziny pracy (np. Pn-Pt_8-16):*Pn-Pt_8-16*

Dodano pomyslnie!

Dodac kolejnego? (1 - Tak, 0 - Zakoncz):*0*

ZARZADZANIE LEKARZAMI

1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot

Wybierz opcje:|

ZARZADZANIE LEKARZAMI

1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot

Wybierz opcje:*2*

ID: 1 | asd Semen Rotbart | Lekarz

Czy chcesz odsortowac dane? (1-Tak, Inne-Nie)

```

ZARZADZANIE LEKARZAMI
1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot
Wybierz opcje:3

Usuwanie lekarza
Podaj id lekarza do usuniecia:
1

Lekarz o id 1 zostal usuniety

ZARZADZANIE LEKARZAMI
1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot
Wybierz opcje:

```

```

Lekarz o id 1 zostal usuniety

ZARZADZANIE LEKARZAMI
1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot
Wybierz opcje:4

Baza pusta;

ZARZADZANIE LEKARZAMI
1. Dodaj lekarzy
2. Pokaz lekarzy
3. Usuwanie lekarzy
4. Wyszukiwanie lekarzy
0. Powrot
Wybierz opcje:

```

```

Wybierz opcje:0

MENU GLOWNE
1. Lekarze
2. Pacjenci
3. Wizyty
0. Wyjdz z programu
Wybierz opcje:3

ZARZADZANIE WIZYTAMI
1. Dodaj wizyte
2. Pokaz wizyty
3. Usuwanie wizyty
4. Wyszukiwanie wizyty
0. Powrot
Wybierz opcje:|

```

5. Opis użytych algorytmów i najciekawsze fragmenty kodu

Zamiast zwykłych tablic , użyliśmy dynamicznych list jednokierunkowych. Dzięki temu możemy dodawać tylu pacjentów i lekarzy, na ile starczy pamięci RAM w komputerze.

a) System sprawdzania nakładających się wizyt To algorytm, z którego jesteśmy najbardziej zadowoleni. Aby sprawdzić, czy wizyty nie nachodzą na siebie czasowo, przeliczamy format HH MM na minuty od północy. Następnie sprawdzamy, czy przedziały czasowe się krzyżują. Oto fragment naszego kodu z pliku wizyta.c

```
int ha,hb,ma,mb;
sscanf(check->time, "%d:%d", &hb, &mb);
sscanf(temp_time, "%d:%d", &ha, &ma);
int time_b = hb * 60 + mb;
int end_old = time_b + check->duration;
int time_a = ha * 60 + ma;
int end_new = time_a + temp_duration;
if (time_a < end_old && time_b < end_new) {
    conflict = true;
}
```

b) Zarządzanie pamięcią Przy listach wskaźnikowych trzeba bardzo uważać na wycieki pamięci. Podczas usuwania elementu użyliśmy dwóch wskaźników , aby najpierw bezpiecznie przełączyć wskaźnik na kolejny element , a dopiero potem oczyścić stertę funkcją free():

```
if (current != NULL && current->id == target_id) {
    prev->next = current->next;
    free(current);
}
```

c) Szybkie sortowanie i pliki Do sortowania lekarzy użyliśmy wbudowanej funkcji qsort. Jednak żeby nie obciążać procesora kopiowaniem dużych struktur, sortujemy tylko tymczasową tablicę wskaźników, a potem od nowa łączymy elementy. Zapis do plików odbywa się przy pomocy funkcji fprintf przy wyłączaniu programu.

```
Lekarz **tab = malloc(count * sizeof(Lekarz *));
if (tab == NULL) {
    printf("Bład pamieci!\n");
    return;
}

current = *head;
for (int i = 0; i < count; i++) {
```

```

    tab[i] = current;
    current = current->next;
}
qsort(tab, count, sizeof(Lekarz *), compare);
for (int i = 0; i < count - 1; i++) {
    tab[i]->next = tab[i + 1];
}
tab[count - 1]->next = NULL;
*head = tab[0];
for (int i = 0; i < count; i++) {
    printf("ID: %d | %s %s %s | %s\n", tab[i]->id, tab[i]->title,
tab[i]->name, tab[i]->surname, tab[i]->typ);
}
free(tab);

```

d) Dynamiczne dodawanie do listy Zarządzanie listą wymaga odpowiedniej alokacji pamięci. Poniższy fragment z funkcji add_lekarz pokazuje, jak bezpiecznie rezerwujemy pamięć funkcją malloc i jak algorytm iteruje po liście, by znaleźć jej koniec w celu nadania unikalnego, kolejnego numeru ID:

```

while (choice != 0) {
    Lekarz *nowy = (Lekarz*)malloc(sizeof(Lekarz));
    if (nowy == NULL) {
        printf("Blad: Brak pamieci!\n");
        return;
    }
    nowy->next = NULL;

    if (*head == NULL) {
        nowy->id = 1;
    } else {
        Lekarz *temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        nowy->id = temp->id + 1;
    }

    if (*head == NULL) {
        *head = nowy;
    } else {
        Lekarz *temp = *head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = nowy;
    }
}

```

e) Algorytm wyszukiwania liniowego Wyszukiwanie konkretnych rekordów zaimplementowaliśmy za pomocą wyszukiwania liniowego o złożoności czasowej. Algorytm przechodzi przez całą listę węzłów po węzle, aż natrafi na szukany ciąg znaków, używając funkcji strcmp:

```
printf("Wyszukiwanie lekarza\n");
char search[20];
printf("Wprowadz numer PESEL lekarza: \n");
scanf("%19s", search);
Lekarz *current = *head;
bool found = false;
while (current != NULL) {
    if (strcmp(current->pesel, search) == 0) {
        printf("ID: %d\n", current->id);
        printf("Name: %s\n", current->name);
        printf("Surname: %s\n", current->surname);
        printf("Pesel: %s\n", current->pesel);
        printf("PWZ: %s\n", current->pwz);
        printf("Tytul: %s\n", current->title);
        printf("Specjalizacja: %s\n", current->typ);
        printf("Email: %s\n", current->email);
        printf("Phone: %s\n", current->phone);
        printf("Godziny pracy: %s\n", current->hours);
        found = true;
        int edit;
        printf("Czy chcesz edytowac dane lekarza? (1-Tak, 0-Nie)");
        scanf("%d", &edit);
        if (edit == 1) {
            int field = -1;
            while (field != 0) {
                printf("1.ID: %d\n", current->id);
                printf("2.Name: %s\n", current->name);
                printf("3.Surname: %s\n", current->surname);
                printf("4.PWZ: %s\n", current->pwz);
                printf("5.Email: %s\n", current->email);
                printf("6.Phone: %s\n", current->phone);
                printf("7.Tytul: %s\n", current->title);
                printf("8.Specjalizacja: %s\n", current->typ);
                printf("9.Godziny pracy: %s\n", current->hours);
                printf("0. Wyjscie\n");
                scanf("%d", &field);
                switch (field) {
                    case 1: {
                        int assured;
                        printf("Zmiana id nie jest polecana, czy na pewno chcesz kontynuowac?(Wprowadz 1 dla potwierdzenia)\n");
                        scanf("%d", &assured);
                        if (assured == 1) {
                            printf("Wprowadz nowe id:\n");
                            scanf("%d", &current->id);
                            break;
                        }
                    }
                }
            }
        }
    }
    current = current->next;
}
```

```

        }
        else {
            printf("Madry wybor\n");
            break;
        }
    }
    case 2:
        printf("Wprowadz nowe imie:\n");
        scanf("%s", current->name);
        break;
    case 3:
        printf("Wprowadz nowe nazwisko:\n");
        scanf("%s", current->surname);
        break;
    case 4:
        printf("Wprowadz nowe PWZ:\n");
        scanf("%s", current->pwz);
        break;
    case 5:
        printf("Wprowadz nowy email:\n");
        scanf("%s", current->email);
        break;
    case 6:
        printf("Wprowadz nowy numer telefonu:\n");
        scanf("%s", current->phone);
        break;
    case 7:
        printf("Wprowadz nowy tytul:\n");
        scanf("%s", current->title);
        break;
    case 8:
        printf("Wprowadz nowa specjalizacje:\n");
        scanf("%s", current->typ);
        break;
    case 9:
        printf("Wprowadz nowe godziny pracy:\n");
        scanf("%s", current->hours);
        break;
    case 0:
        printf("Zakonczono edycje.\n");
        break;
    default:
        printf("Nieznana opcja!\n");
    }
}

}

}

current = current->next;
}
if (!found) {
    printf("Nie odnaleziono lekarza");
}

```


f) Wyświetlanie bazy danych Wypisywanie wszystkich rekordów na ekran wymaga przejścia przez całą listę jednokierunkową od wskaźnika head aż do napotkania wartości NULL . Zabezpieczyliśmy również ten algorytm przed próbą wyświetlenia pustej bazy:

```
Lekarz *current = head;
int choice = 1;
if (current == NULL) {
    printf("Baza pusta\n");
    return;
}
while (current != NULL) {
    printf("ID: %d | %s %s %s | %s\n", current->id,
current->title, current->name, current->surname, current->typ);
    current = current->next;
}
```

6. Krótkie podsumowanie i co można poprawić

Udało nam się zrealizować w 100% plan, który założyliśmy na początku semestru. Projekt posiada wszystkie wymagane funkcje. Program nie crashuje się przy podstawowych operacjach. Czego zabrakło Co byśmy dodali w przyszłości:

- Lepsze zabezpieczenie funkcji scanf obecnie, jeśli użytkownik wpisze literę zamiast cyfry ID, program może wpaść w nieskończoną pętlę. Przydałaby się mocniejsza walidacja głupot wpisywanych przez użytkownika.
- Szyfrowanie plików tekstowych, żeby dane przychodziły nie leżały jawnym tekstem na dysku.
- Jakies proste statystyki generowane do pliku, np który lekarz ma najwięcej umówionych wizyt.

7. Podział pracy w zespole

Podzieliliśmy się pracą tak, żeby każdy miał kontakt ze wskaźnikami i listami:

- **Semen Rotbart:** Zaprojektowanie struktur danych, napisanie algorytmów list jednokierunkowych oraz napisanie całego modułu odpowiedzialnego za pliki .txt.
- **Andrii Riabchenko:** Zaprojektowanie modułu Wizyty, zrobienie filtrów, wyszukiwarek, podpięcie qsort do list oraz połączenie tego wszystkiego w działające menu konsolowe.